

Convolutional Neural Network (CNN)

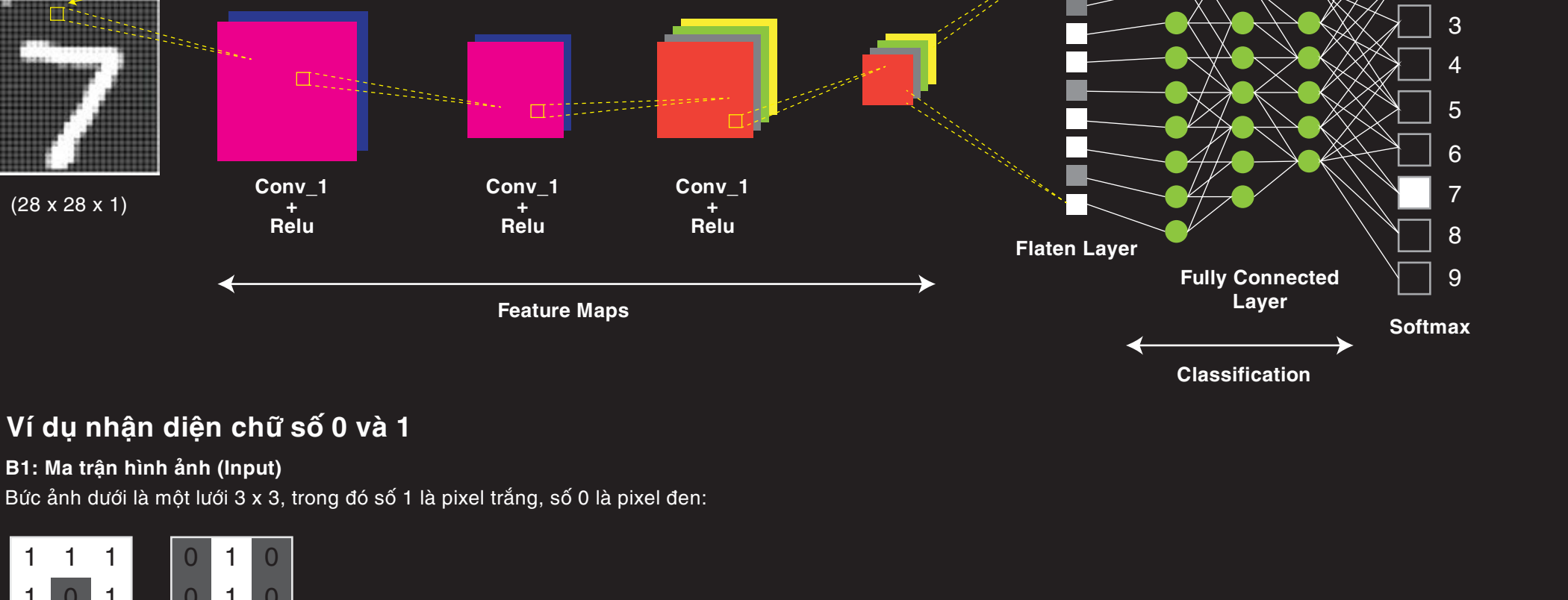
CNN là một loại mạng nơ-ron được thiết kế đặc biệt để "bắt chước" cách hệ thống thị giác của con người hoạt động. Thay vì nhìn vào từng pixel riêng lẻ, CNN nhìn vào các mẫu hình (patterns). Nó bắt đầu bằng việc nhận diện các đường kẻ, cạnh sắc, sau đó ghép chúng lại để thấy hình tròn, hình vuông, và cuối cùng là nhận diện được khuôn mặt hay đồ vật.

CNN hoạt động dựa trên 3 thành phần cốt lõi:

- Lớp Tích chập (Convolutional Layer) - "Cửa sổ quét".
 - Đây là bộ phận quan trọng nhất. Hãy tưởng tượng bạn cầm một chiếc kính lúp nhỏ quét qua từng phần của bức ảnh.
 - Chiếc kính lúp này (gọi là Filter hay Kernel) sẽ trích xuất các đặc trưng như: cạnh ngang, cạnh dọc, màu sắc...
- Lớp Kích hoạt (ReLU Layer) - "Bộ lọc thông tin".
 - Sau khi quét xong, dữ liệu sẽ đi qua hàm ReLU. Nhiệm vụ của nó rất đơn giản: giữ lại những thông tin quan trọng (giá trị dương) và loại bỏ những nhiễu không cần thiết (chuyển các giá trị âm về 0). Điều này giúp mạng học nhanh và hiệu quả hơn.
- Lớp Gộp (Pooling Layer) - "Tóm tắt ý chính".
 - Để giảm bớt khối lượng tính toán, CNN sử dụng Pooling (thường là Max Pooling).
 - Nó sẽ nhìn vào một vùng ảnh và chỉ giữ lại giá trị nổi bật nhất.
 - Việc này giống như việc bạn nhìn một bức ảnh từ xa: bạn không cần thấy từng sợi lông của con mèo nhưng vẫn biết đó là con mèo.

Nó giúp mạng trở nên "lazier" hơn với các biến đổi nhỏ (như ảnh bị lệch hay xoay nhẹ).

Bản chất của CNN là dạy máy tính cách "nhìn" từ những chi tiết nhỏ nhất (đường nét, góc cạnh) rồi từ đó ghép chúng lại để hiểu được toàn bộ bức tranh (một khuôn mặt, một con vật, một chiếc xe).



Ví dụ nhận diện chữ số 0 và 1

B1: Ma trận hình ảnh (Input)
Bức ảnh dưới là một lưới 3 x 3, trong đó số 1 là pixel trắng, số 0 là pixel đen:

1	0	1
1	0	1
1	1	1

0	1	0
0	1	0
0	1	0

Input (X) = $\begin{bmatrix} 0 & 1 & 0 \\ 0 & 1 & 0 \\ 0 & 1 & 0 \end{bmatrix}$

B2: Tích chập (Convolution)
Chúng ta dùng một bộ lọc (Kernel) kích thước 2 x 2 được thiết kế để dò "nét thẳng dọc"
Kernel = 2 x 2
Stride = 1

Filter (W) = $\begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$, Bias (b) = 0

Trượt bộ lọc W trên ảnh X. Chúng ta sẽ có 4 vị trí trượt.
1. Vị trí 1 (Góc trên trái): (-1 x 0) + (1 x 1) + (-1 x 0) + (1 x 1) = 2
2. Vị trí 2 (Góc trên phải): (-1 x 1) + (1 x 1) + (-1 x 1) + (1 x 0) = -2
3. Vị trí 3 (Góc dưới trái): (-1 x 0) + (1 x 1) + (-1 x 0) + (1 x 1) = 2
4. Vị trí 4 (Góc dưới phải): (-1 x 1) + (1 x 1) + (-1 x 1) + (1 x 0) = -2

Kết quả sau Convolution: $\begin{bmatrix} 2 & -2 \\ 2 & -2 \end{bmatrix}$

B3: Hàm kích hoạt (ReLU)
Áp dụng f(x) = max(0, x). Mọi số âm biến thành 0.

Kết quả sau ReLU: $\begin{bmatrix} 2 & 0 \\ 2 & 0 \end{bmatrix}$

B4: Max Pooling
Áp dụng một lớp Max Pooling để lấy giá trị lớn nhất trong cả vùng 2 x 2 đó
max(2, 0, 2, 0) = 2
-> Ma trận 2 x 2 bây giờ bị thu nhỏ lại chỉ còn duy nhất một con số: [2]
Tuy nhiên chúng ta không sử dụng max pooling trong trường hợp này được vì:
1. Vấn đề kích thước:
- Trong thực tế: Ảnh đầu vào thường là 224 x 224 hoặc lớn hơn. Nếu không có Max Pooling để nén kích thước xuống, số lượng tham số ở lớp cuối cùng (Flatten) sẽ bùng nổ lên hàng triệu, khiến máy tính "treo" ngay lập tức.
- Trong ví dụ 3x3: Sau khi tích chập 2 x 2 lần thì còn lại ma trận 2 x 2 (4 con số). Việc nén 4 con số xuống 1 con số bằng Max Pooling không giúp máy tính nhanh hơn bao nhiêu. Nó giống như việc dùng một chiếc xe tải 10 tấn chỉ để chở một bộ bao gạo vậy.
2. Mất mát thông tin (Information Loss):
Với một ma trận quá nhỏ, mỗi pixel đều cực kỳ quý giá.
- Khi em dùng Max Pooling 2 x 2 trên ma trận 2 x 2, em đang vứt bỏ 75% dữ liệu (chỉ giữ lại 1 số lớn nhất trong 4 số).
- Trong không gian 3x3, hành động này có thể làm mất đi các chi tiết quan trọng giúp phân biệt các số có hình dáng gần giống nhau.
3. Khi nào nên dùng Max Pooling cho ảnh nhỏ?
Người ta chỉ dùng Max Pooling khi muốn đạt được sự "Bất biến về vị trí" (Translation Invariance).
Nếu em muốn dò số 1 nằm ở sát mép trái hay sát mép phải của khung 3x3 thì máy vẫn nhận ra, lúc đó Max Pooling mới có tác dụng "gom" đặc trưng đó lại.

Bước 5: Làm phẳng & Tính các lớp kết nối đầy đủ (Flatten & Softmax)

Sau khi làm phẳng, ta có có vector [2, 0, 2, 0] và đi qua lớp kết nối để có 2 giá trị logit cuối cùng.
Tại đây, mỗi "nút" đầu ra (một nút cho Số 0, một nút cho Số 0) sẽ có một bộ Trọng số (Weights) riêng. Các trọng số này chính là thứ mà mạng AI đã "học" được sau hàng triệu lần luyện tập.
Weights cho Số 1 (W1): [1.2, -0.5, 1.3, -0.1]
Bias (b1): 0
Weights cho Số 0 (W0): [0.2, 0.1, 0.3, 0.1]
Bias (b0): 0
Phép tính tại lớp kết nối đầy đủ
Logit cho số 1 = (1.2 x 2) + (-0.5 x 0) + (1.3 x 2) + (-0.1 x 0) + 0 = 5.0
Logit cho số 0 = (0.2 x 2) + (0.1 x 0) + (0.3 x 2) + (0.1 x 0) = 1.0

Bước 6: Softmax
Xác suất cho số 1: $\frac{e^{5.0}}{e^{5.0} + e^{1.0}} = 0.982$
Xác suất cho số 0: $\frac{e^{1.0}}{e^{5.0} + e^{1.0}} = 0.017$
Bước 7: Thay đổi dấu vào
Nếu ta thay đổi ảnh Input thành một nét ngang như này:

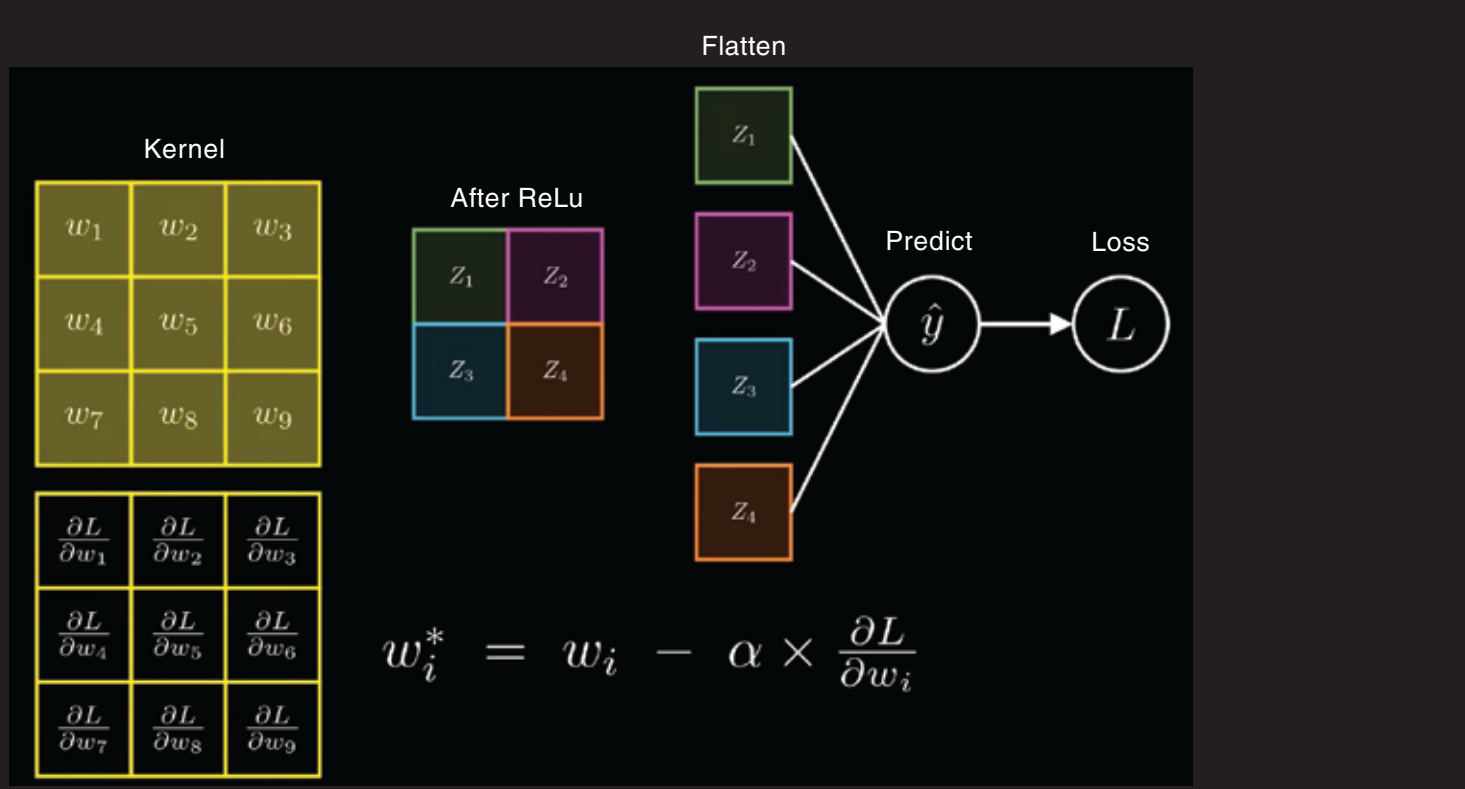
0	0	0
1	1	1
0	0	0

 và dùng bộ lọc kernel để dò nét dọc như trên $\text{Filter (W)} = \begin{bmatrix} -1 & 1 \\ -1 & 1 \end{bmatrix}$, Bias (b) = 0

Lúc này phép tích chập xuất ra kết quả là ma trận 2x2 nhưng các giá trị đều bằng 0.
Vi khi ta đưa vào một nét ngang ([1, 1, 1] ở giữa), tại mọi vị trí trượt của bộ lọc 2 x 2, ta sẽ luôn nhân (-1 x 1) + (1 x 1) = 0.
Các con số tự triệt tiêu lẫn nhau vì bộ lọc không tìm thấy "nét đúng" nào trong một cấu trúc nằm ngang.
Bộ lọc này có cột bên trái là số âm, cột bên phải là số dương. Trong toán học, kiểu thiết kế này gọi là bộ dò cạnh đứng (Vertical Edge Detector).
Nó được tạo ra để tìm kiếm sự thay đổi về cường độ điểm ảnh theo chiều ngang (từ trái qua phải).
Trong kiến trúc CNN: Bộ lọc là những "chuyên gia"
Trong một mạng CNN thực tế lớp đầu tiên không chỉ có 1 bộ lọc mà nó có hàng chục bộ lọc khác nhau.
Số có bộ lọc chuyên tìm nét đứng (như bộ lọc ta vừa tính), bộ lọc chuyên tìm nét ngang, bộ lọc tìm góc chéo, v.v.
Kết quả toàn số 0 có nghĩa là: "Chuyên gia dò nét đứng" báo cáo rằng: "Tôi không thấy nét nào thuộc chuyên môn của tôi ở đây cả"
Sự khác biệt giữa Số 1 và Số 0:
- Nếu đầu vào là Số 1 (nét đứng): Bộ lọc nét đứng sẽ cho giá trị cao (tạo ra Logit lớn cho số 1).
- Nếu đầu vào là Số 0 (có cả nét ngang và đứng): Bộ lọc nét đứng sẽ chỉ "sáng" lên ở hai bên cạnh của số 0, còn bộ lọc nét ngang sẽ "sáng" lên ở đỉnh và đáy của số 0.
Mạng nơ-ron phân biệt các vật thể dựa trên sự kết hợp của các bộ lọc này. Khi nét ngang đi qua bộ lọc nét đứng và ra kết quả bằng 0, mạng nơ-ron sẽ hiểu rằng: "Đây chắc chắn không phải là một nét đứng". Đó là một thông tin cực kỳ hữu ích để loại trừ khả năng đó là số 1.

Nhưng làm sao để các bộ lọc "ngẫu nhiên" ban đầu tự biến thành "bộ lọc nét đứng" hay "bộ lọc nét ngang"? Máy tính sẽ dùng Gradient Descent.
Nếu nó đoán sai, nó sẽ điều chỉnh các con số trong bộ lọc W từng chút một. Khoảng cách mỗi lần điều chỉnh đó chính là Learning Rate.
Nếu learning rate quá lớn: Máy tính điều chỉnh "giật cục", để làm hỏng các kiến thức đã học.
Nếu learning rate quá nhỏ: Máy tính học rất chậm, mất cả ngày mới nhận diện được một nét thẳng.

Backpropagation



Bây giờ chúng ta cần tính đạo hàm riêng của Hàm mất mát (Loss) đối với từng trọng số (w)
 $\frac{\partial L}{\partial w_1} = \frac{\partial z_1}{\partial w_1} \cdot \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial L}{\partial \hat{y}}$

- $\frac{\partial L}{\partial \hat{y}}$: Sai số thay đổi thế nào theo dự đoán đầu ra.
- $\frac{\partial \hat{y}}{\partial z_1}$: Dự đoán đầu ra thay đổi thế nào theo giá trị Logit (ví dụ con số 8.65 của Espresso trong ảnh của em).
- $\frac{\partial z_1}{\partial w_1}$: Giá trị Logit thay đổi thế nào theo trọng số của sợi dây kết nối.

Ở trên, y mũ nhân chéo nên được triệt tiêu và rút gọn thành biểu thức dưới đây
 $\frac{\partial L}{\partial w_1} = \frac{\partial z_1}{\partial w_1} \cdot \frac{\partial L}{\partial z_1}$
Vi trong số w1 ảnh hưởng đến tất cả 4 phép tính đầu ra z1, z2, z3, z4 nên nếu kết quả cuối cùng bị sai (Loss cao), chúng ta không thể chỉ đổ lỗi cho z1
- Một phần lỗi của L đến từ z1, mà z1 lại do w1 góp sức.
- Một phần lỗi khác của L đến từ z2, mà z2 cũng có "dấu vân tay" của w1 trong đó.
Đó là lý do tại sao công thức đạo hàm phải cộng dồn tất cả các nhánh lại và chúng ta sẽ có phương trình này cho đạo hàm riêng của tổn thất theo w1

$\frac{\partial L}{\partial w_1} = \frac{\partial z_1}{\partial w_1} \frac{\partial L}{\partial z_1} + \frac{\partial z_2}{\partial w_1} \frac{\partial L}{\partial z_2} + \frac{\partial z_3}{\partial w_1} \frac{\partial L}{\partial z_3} + \frac{\partial z_4}{\partial w_1} \frac{\partial L}{\partial z_4}$

Vi vậy điều tương tự cũng được áp dụng cho tất cả các trọng số khác, cho chúng ta công thức tổng quát này
 $\frac{\partial L}{\partial w_i} = \frac{\partial z_1}{\partial w_i} \frac{\partial L}{\partial z_1} + \frac{\partial z_2}{\partial w_i} \frac{\partial L}{\partial z_2} + \frac{\partial z_3}{\partial w_i} \frac{\partial L}{\partial z_3} + \frac{\partial z_4}{\partial w_i} \frac{\partial L}{\partial z_4}$

Bây giờ hãy xem xét các số hạng mà ta lấy đạo hàm riêng của L (loss) theo w1, và tính toán bằng cách sử dụng các phương trình z1,z2,... trước đó.
- Các cụm không chứa w1 (như w2 x a2, w3 x a3, w4 x a6,...) đều có đạo hàm bằng 0 vì chúng là hằng số
- Đạo hàm của (w1 x a1) theo w1 chính là bằng a1
Vậy nên đạo hàm của z1 theo w1 là a1, đạo hàm của z2 theo w1 là a3,...

$\frac{\partial L}{\partial w_1} = \underbrace{\frac{\partial z_1}{\partial w_1} \frac{\partial L}{\partial z_1}}_{a_1} + \underbrace{\frac{\partial z_2}{\partial w_1} \frac{\partial L}{\partial z_2}}_{a_3} + \underbrace{\frac{\partial z_3}{\partial w_1} \frac{\partial L}{\partial z_3}}_{a_{11}} + \underbrace{\frac{\partial z_4}{\partial w_1} \frac{\partial L}{\partial z_4}}_{a_{13}}$
 $z_1 = w_1 \times a_1 + w_2 \times a_2 + w_3 \times a_3 + w_4 \times a_6 \dots w_9 \times a_{13}$
 $z_2 = w_1 \times a_3 + w_2 \times a_4 + w_3 \times a_5 + w_4 \times a_8 \dots w_9 \times a_{15}$
 $z_3 = w_1 \times a_{11} + w_2 \times a_{12} + w_3 \times a_{13} + w_4 \times a_{16} \dots w_9 \times a_{23}$
 $z_4 = w_1 \times a_{13} + w_2 \times a_{14} + w_3 \times a_{15} + w_4 \times a_{18} \dots w_9 \times a_{25}$

$\frac{\partial L}{\partial w_1} = a_1 \frac{\partial L}{\partial z_1} + a_3 \frac{\partial L}{\partial z_2} + a_{11} \frac{\partial L}{\partial z_3} + a_{13} \frac{\partial L}{\partial z_4}$
 $z_1 = w_1 \times a_1 + w_2 \times a_2 + w_3 \times a_3 + w_4 \times a_6 \dots w_9 \times a_{13}$
 $z_2 = w_1 \times a_3 + w_2 \times a_4 + w_3 \times a_5 + w_4 \times a_8 \dots w_9 \times a_{15}$
 $z_3 = w_1 \times a_{11} + w_2 \times a_{12} + w_3 \times a_{13} + w_4 \times a_{16} \dots w_9 \times a_{23}$
 $z_4 = w_1 \times a_{13} + w_2 \times a_{14} + w_3 \times a_{15} + w_4 \times a_{18} \dots w_9 \times a_{25}$

Tương tự, đạo hàm của L (loss) theo w2 và các trọng số còn lại cũng vậy
 $\frac{\partial L}{\partial w_2} = a_2 \frac{\partial L}{\partial z_1} + a_4 \frac{\partial L}{\partial z_2} + a_{12} \frac{\partial L}{\partial z_3} + a_{14} \frac{\partial L}{\partial z_4}$
 $z_1 = w_1 \times a_1 + w_2 \times a_2 + w_3 \times a_3 + w_4 \times a_6 \dots w_9 \times a_{13}$
 $z_2 = w_1 \times a_3 + w_2 \times a_4 + w_3 \times a_5 + w_4 \times a_8 \dots w_9 \times a_{15}$
 $z_3 = w_1 \times a_{11} + w_2 \times a_{12} + w_3 \times a_{13} + w_4 \times a_{16} \dots w_9 \times a_{23}$
 $z_4 = w_1 \times a_{13} + w_2 \times a_{14} + w_3 \times a_{15} + w_4 \times a_{18} \dots w_9 \times a_{25}$

Cuối cùng, ta sẽ có 9 công thức đạo hàm cho 9 trọng số (từ w1 đến w9)
 $\frac{\partial L}{\partial w_1} = a_1 \frac{\partial L}{\partial z_1} + a_3 \frac{\partial L}{\partial z_2} + a_{11} \frac{\partial L}{\partial z_3} + a_{13} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_2} = a_2 \frac{\partial L}{\partial z_1} + a_4 \frac{\partial L}{\partial z_2} + a_{12} \frac{\partial L}{\partial z_3} + a_{14} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_3} = a_3 \frac{\partial L}{\partial z_1} + a_5 \frac{\partial L}{\partial z_2} + a_{13} \frac{\partial L}{\partial z_3} + a_{15} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_4} = a_6 \frac{\partial L}{\partial z_1} + a_8 \frac{\partial L}{\partial z_2} + a_{16} \frac{\partial L}{\partial z_3} + a_{18} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_5} = a_7 \frac{\partial L}{\partial z_1} + a_9 \frac{\partial L}{\partial z_2} + a_{17} \frac{\partial L}{\partial z_3} + a_{19} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_6} = a_8 \frac{\partial L}{\partial z_1} + a_{10} \frac{\partial L}{\partial z_2} + a_{18} \frac{\partial L}{\partial z_3} + a_{20} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_7} = a_{11} \frac{\partial L}{\partial z_1} + a_{13} \frac{\partial L}{\partial z_2} + a_{21} \frac{\partial L}{\partial z_3} + a_{23} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_8} = a_{12} \frac{\partial L}{\partial z_1} + a_{14} \frac{\partial L}{\partial z_2} + a_{22} \frac{\partial L}{\partial z_3} + a_{24} \frac{\partial L}{\partial z_4}$
 $\frac{\partial L}{\partial w_9} = a_{13} \frac{\partial L}{\partial z_1} + a_{15} \frac{\partial L}{\partial z_2} + a_{23} \frac{\partial L}{\partial z_3} + a_{25} \frac{\partial L}{\partial z_4}$

Nếu đạo hàm của z theo w trả lời cho câu hỏi "Trọng số này đóng góp bao nhiêu vào giá trị kết quả"
Thì đạo hàm của L theo z trả lời cho câu hỏi "Nốt z này đã làm sai lệch kết quả cuối cùng bao nhiêu?"
Theo quy tắc chuỗi (chain rule) ta có công thức:

$\frac{\partial L}{\partial z_1} = \frac{\partial \hat{y}}{\partial z_1} \cdot \frac{\partial L}{\partial \hat{y}}$

Để giải đạo hàm của L theo z1 ta sẽ phải thực hiện hai phép tính đạo hàm riêng lẻ:

- Bước A: $\frac{\partial L}{\partial \hat{y}}$ (Lỗi từ hàm Loss)**
 - Nếu ta dùng hàm sai số bình phương $L = \frac{1}{2} (y_{true} - \hat{y})^2$, đạo hàm sẽ là: $-(y_{true} - \hat{y})$.
 - Đây là con số cho biết sự chênh lệch giữa thực tế và dự đoán.
- Bước B: $\frac{\partial \hat{y}}{\partial z_1}$ (Đạo hàm hàm kích hoạt)**
 - Vì $\hat{y} = \text{Softmax}(z)$, nên đây là đạo hàm của hàm Softmax (hoặc Sigmoid/ReLU tùy lớp).
 - Nó cho biết một thay đổi nhỏ ở giá trị thô z sẽ làm thay đổi xác suất đầu ra $\hat{y}$ bao nhiêu.

Máy tính những tính từng dòng công thức như con người. Nó dùng một thủ thuật ma trận siêu thông minh được vẽ ở hàng dưới cùng.
Thay vì hình lật nhào từng cái L theo w1, w2,..., nó làm như sau:

- Lấy khung ảnh thứ 1 (Xanh lá):** Chứa các pixel tạo ra z1. Nó lấy **toàn bộ ma trận này nhân với một con số duy nhất** là tín hiệu lỗi $\frac{\partial L}{\partial z_1}$.
 - Lấy khung ảnh thứ 2 (Tím):** Nhân toàn bộ với tín hiệu lỗi $\frac{\partial L}{\partial z_2}$.
 - Tương tự cho khung Xanh dương và Cam.**
 - Cộng dồn (Đấu +):** Nó lấy 4 ma trận kết quả ở trên đặt chồng lên nhau và cộng các vị trí tương ứng lại.
- Kết quả:** BÙM! Tổng của 4 ma trận đó sinh ra đúng một **ma trận vàng 3 x 3** ở ngoài cùng bên phải. Và ma trận này chứa **chính xác 9 kết quả đạo hàm** mà ta cần tìm (từ $\frac{\partial L}{\partial w_1}$ đến $\frac{\partial L}{\partial w_9}$).

